

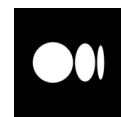
Natural Language Processing (NLP)



istdatascience.com | info@istdsa.com



tr.linkedin.com/school/istanbul-data-science-academy



medium.com/istanbuldatascienceacademy

STRUCTURED VS UNSTRUCTURED VERİLER

Şu ana kadar tüm çalışmalarımızı **structured** veriler üzerinden gerçekleştirdik. Ancak günümüzdeki veri kaynakları sadece bu tablo yapılarıyla sınırlı değil!

Günümüzde birçok şirket, müşterilerini daha iyi tanıyabilmek için **unstructured** veri kaynaklarıyla çalışmaya başladı. Peki ne bunlar?



Mark Cuban
@mcuban

Follow

If you only do what you have to do, you will always have to do what you are told to do.

6:54 PM - 7 Aug 2017

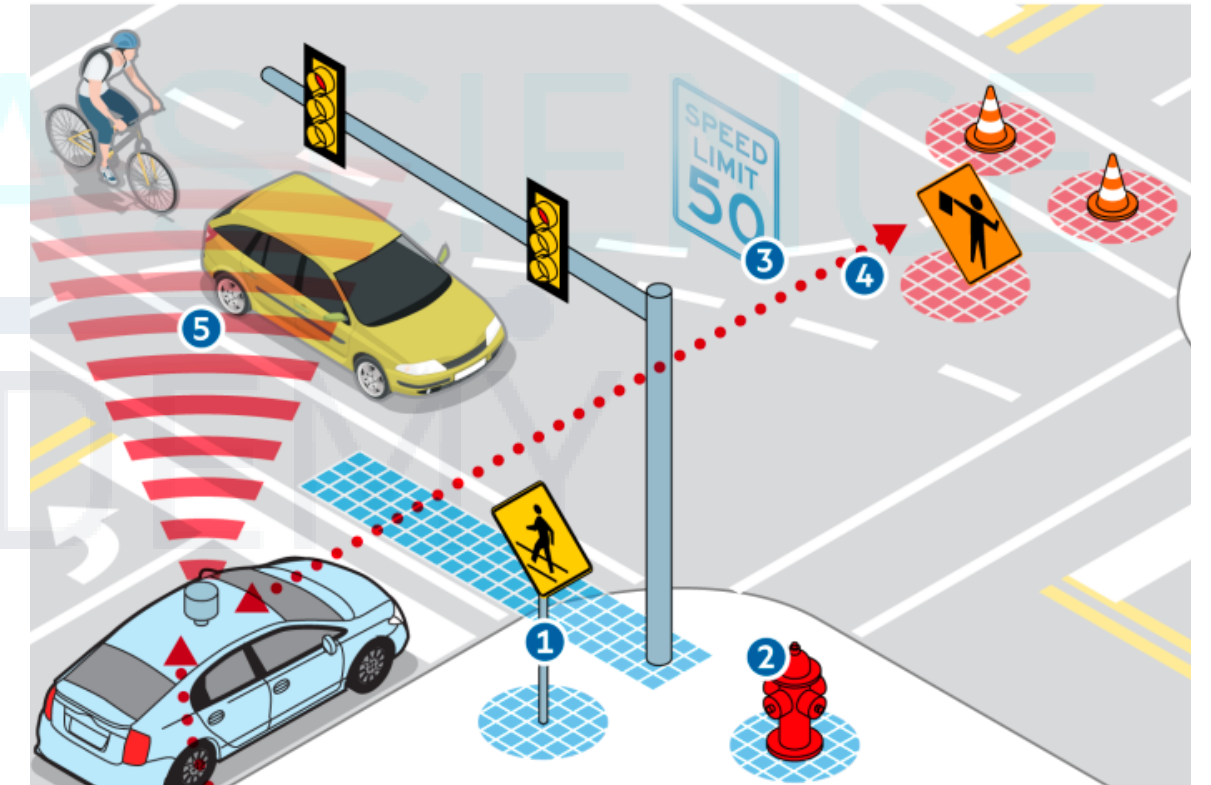
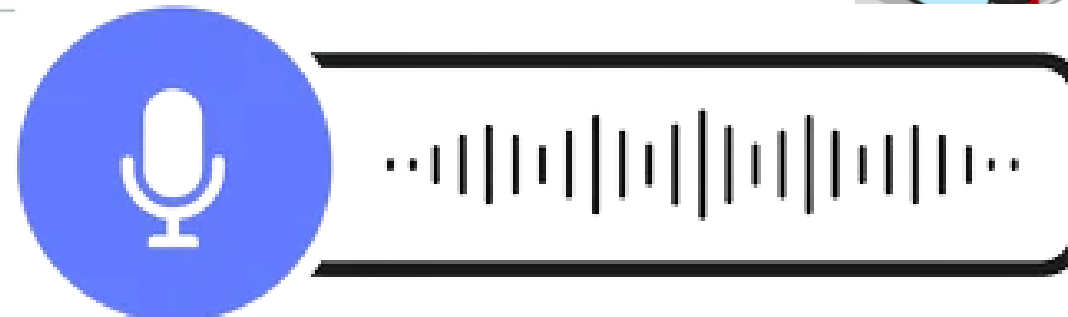
5,240 Retweets 11,407 Likes



247

5.2K

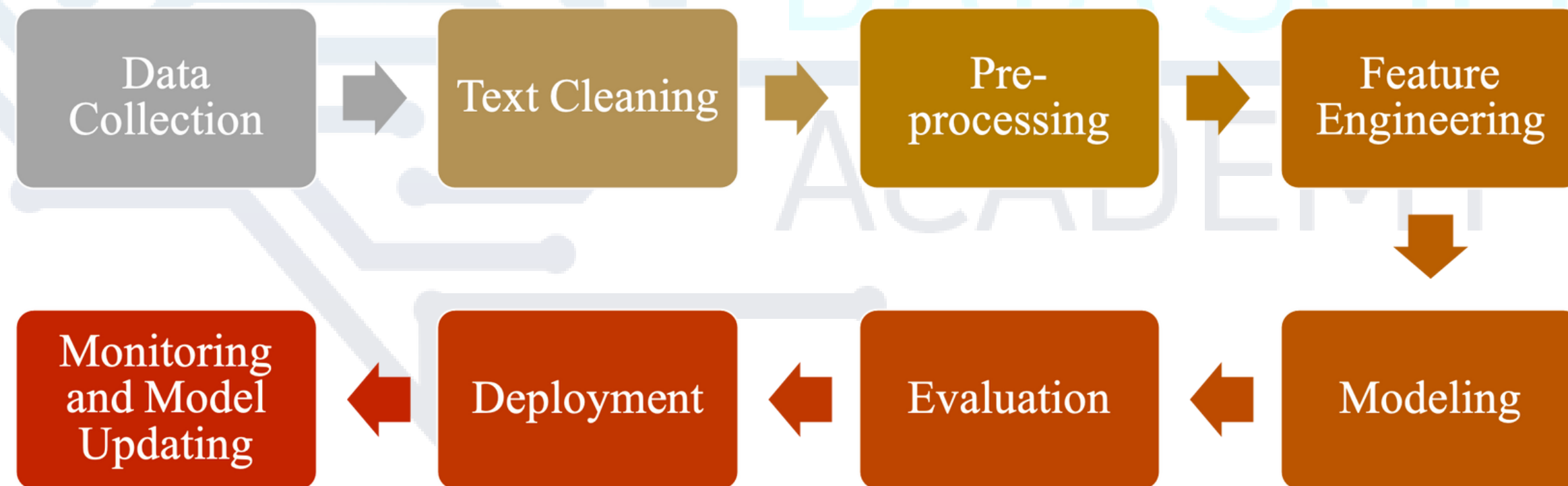
11.4K



UNSTRUCTURED VERİLER

Bize sağladıkları bilgilerin yanında, unstructured verilerle çalışmak **daha zordur** ve şu ana kadar gördüğümüz **SQL veritabanlarında tutulamazlar**.

Amacımız özellikle metin verilerinden anlam çıkarmak olduğunda, metin verilerimizin içerisindeki yer alan gürültülerden kurtulmamız gerekir.



NLP Pipeline

Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from the store. Should I pick up some black-eyed peas as well?

- Gerçekten bu metindeki tüm kelimelere ihtiyacım var mı?
- Noktalama işaretlerini de kullanmam gerekiyor mu?
- Büyük-küçük harf sorunu?
- Yazım yanlışlarını nasıl düzeltebilirim?
- Aynı kelime kökünün farklı çekimleri? Ya ikisi de aynı anlama geliyorsa?
- (Yemek yapmayı **seviyorum** / Yemek yapmayı **severim**)

Elimizdeki metin verisini küçük ve anlamlı parçalara ayırma işlemi. Peki nasıl parçalar?

- **Cümleler**
- **Kelimeler**
- **N-Grams**
- **Regular Expression kullanılarak oluşturulan yapılar**

TOKENIZATION - KELİME

INPUT

```
from nltk.tokenize import word_tokenize
```

```
my_text = "Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from  
the store. Should I pick up some black-eyed peas as well?"
```

```
print(word_tokenize(my_text))
```

OUTPUT

```
['Hi', 'Mr.', 'Smith', '!', 'I', '', 'm', 'going', 'to', 'buy', 'some',  
'vegetables', '(', 'tomatoes', 'and', 'cucumbers', ')', 'from', 'the', 'store',  
'.', 'Should', 'I', 'pick', 'up', 'some', 'black-eyed', 'peas', 'as', 'well',  
'?']
```

TOKENIZATION - CÜMLE

INPUT

```
from nltk.tokenize import sent_tokenize
```

```
my_text = "Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from  
the store. Should I pick up some black-eyed peas as well?"
```

```
print(sent_tokenize(my_text))
```

OUTPUT

```
['Hi Mr. Smith!',  
'I'm going to buy some vegetables (tomatoes and cucumbers) from the store.',  
'Should I pick up some black-eyed peas as well?']
```

TOKENIZATION - N-GRAMS

INPUT

```
from nltk.util import ngrams

my_words = word_tokenize(my_text) # This is the list of all words
twograms = list(ngrams(my_words, 2)) # This is for two-word combos, but can pick any n
print(twograms)
```

OUTPUT

```
[('Hi', 'Mr.'), ('Mr.', 'Smith'), ('Smith', '!'), ('!', 'I'), ('I', ''), ('', 'm'), ('m', 'going'), ('going', 'to'), ('to', 'buy'), ('buy', 'some'), ('some', 'vegetables'), ('vegetables', '('), ('(', 'tomatoes'), ('tomatoes', 'and'), ('and', 'cucumbers'), ('cucumbers', ')'), (')', 'from'), ('from', 'the'), ('the', 'store'), ('store', '.'), ('.', 'Should'), ('Should', 'I'), ('I', 'pick'), ('pick', 'up'), ('up', '1/2'), ('1/2', 'lb'), ('lb', 'of'), ('of', 'black-eyed'), ('black-eyed', 'peas'), ('peas', 'as'), ('as', 'well'), ('well', '?')]
```


TOKENIZATION - REGULAR EXPRESSIONS

INPUT

```
from nltk.tokenize import RegexpTokenizer

# RegexpTokenizer with whitespace delimiter
whitespace_tokenizer = RegexpTokenizer("\s+", gaps=True)
print(whitespace_tokenizer.tokenize(my_text))
```

OUTPUT

```
['Hi', 'Mr.', 'Smith!', 'I'm', 'going', 'to', 'buy', 'some', 'vegetables',  
'(tomatoes', 'and', 'cucumbers)', 'from', 'the', 'store.', 'Should', 'I', 'pick',  
'up', 'some', 'black-eyed', 'peas', 'as', 'well?']
```

TOKENIZATION - REGULAR EXPRESSIONS

INPUT

```
from nltk.tokenize import RegexpTokenizer  
  
# RegexpTokenizer to match only capitalized words  
cap_tokenizer = RegexpTokenizer("[A-Z]+'\w]+'")  
print(cap_tokenizer.tokenize(my_text))
```

OUTPUT

```
['Hi', 'Mr', 'Smith', 'Should']
```

DİĞER METİN VERİSİ İŞLEME YÖNTEMLERİ

Tokenization, metin verileri işlemekte kullandığımız tekniklerden sadece bir tanesidir. Daha önümüzde bir sürü yol var. Peki ne bunlar?

- Gereksiz Karakterler
- Kelime Köklerini Elde Etme
- Kelimeleri Gruplarına Göre Etiketleme
- Yazım Yanlışları & Dil Farklılıkları

Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from the store. Should I pick up some black eyed peas as well?

İlk örneğimize döndüğümüzde, sizce burada hangi karakterlere ihtiyacımız yok?

- Noktalama İşaretleri?
- Büyük Harfler?
- Sayılar?

NOKTALAMA İŞARETLERİNİ KALDIRMA

INPUT

```
import re # Regular expression library
import string

# Replace punctuations with a white space
clean_text = re.sub('[%s]' % re.escape(string.punctuation), ' ', my_text)
clean_text
```

OUTPUT

```
'Hi Mr  Smith  I m going to buy some vegetables  tomatoes and cucumbers  from the
store  Should I pick up 2lbs of black eyed peas as well  '
```


BÜYÜK-KÜÇÜK HARF SORUNU

INPUT

```
clean_text = clean_text.lower()  
clean_text
```

OUTPUT

```
'hi mr smith i m going to buy some vegetables tomatoes and cucumbers from the  
store should i pick up 2lbs of black eyed peas as well '
```

ANLAM İFADE ETMEYEN SAYILAR

INPUT

```
# Removes all words containing digits  
clean_text = re.sub('\w*\d\w*', ' ', clean_text)  
clean_text
```

OUTPUT

```
'hi mr smith i m going to buy some vegetables tomatoes and cucumbers from the  
store should i pick up of black eyed peas as well '
```

GEREKSİZ KELİMELELER (STOP-WORDS)

Hi Mr. Smith! I'm going to buy **some** vegetables (tomatoes and cucumbers) from the store. Should I pick up **some** black-eyed peas as well?

- Yukarıdaki cümleyi tekrar okuyun. En fazla hangi kelime geçmiş?
- Peki bu kelimeler cümlenin ne demek istediğini hakkında bize bir bilgi veriyor mu?

Dolayısıyla bizim mantıklı analiz sonuçlarına ulaşabilmemiz için, **gerçekten bir anlam ifade eden kelimeleri** kullanmamız gerekiyor!

GEREKSİZ KELİMELELER (STOP-WORDS)

INPUT

```
from nltk.corpus import stopwords  
set(stopwords.words('english'))
```

OUTPUT

```
{'but', 'isn', 'under', 'weren', 'those', 'when', 'why', 'few', 'for', 'it', 'of', 'down', 'ma', 'over',  
'd', 'during', 'shouldn', 'did', 'above', 'below', 'myself', 'further', 'very', 'same', 'too', 'does',  
'through', 'from', 'didn', 'whom', 'and', 'am', 'such', 'out', 'or', 'me', 'has', 'will', 'shan', 'on',  
'then', 'here', 't', 'with', 'some', 'what', 'don', 'were', 'an', 'themselves', 'yourselves', 'off',  
'being', 'more', 'they', 'ourselves', 'into', 'my', 'them', 'ain', 'a', 'wouldn', 'itself', 'i', 'hasn',  
'her', 'their', 'mustn', 'our', 'herself', 'where', 'hers', 'once', 'any', 'theirs', 'before', 'most',  
'other', 'n', 'himself', 'his', 'if', 'he', 'each', 'are', 'how', 'couldn', 'ours', 'doing', 'hadn',  
'needn', 'again', 'these', 'wasn', 'nor', 'do', 'just', 'so', 'we', 'there', 'have', 'by', 'o', 'than',  
're', 'while', 'your', 'at', 'him', 'own', 'can', 'you', 'll', 'between', 'been', 'that', 'is', 'she',  
'yours', 'this', 'was', 'be', 'had', 'doesn', 'no', 'because', 'won', 'both', 'to', 'against', 'aren',  
'y', 'after', 'all', 'up', 've', 'should', 'as', 'in', 'the', 'having', 'until', 'who', 'haven', 'only',  
'm', 'yourself', 'about', 's', 'which', 'now', 'mightn', 'its'}
```

GEREKSİZ KELİMELELER (STOP-WORDS)

INPUT

```
my_text = ["Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from  
the store. Should I pick up some black-eyed peas as well?"]
```

```
# Incorporate stop words when creating the count vectorizer
```

```
cv = CountVectorizer(stop_words='english')
```

```
X = cv.fit_transform(my_text)
```

```
pd.DataFrame(X.toarray(), columns=cv.get_feature_names())
```

OUTPUT

	black	buy	cucumbers	eyed	going	hi	mr	peas	pick	smith	store	tomatoes	vegetables
	0	1	1	1	1	1	1	1	1	1	1	1	1

KELİME KÖKLERİNİ ELDE ETME (STEMMING)

INPUT

```
from nltk.stem.lancaster import LancasterStemmer  
  
stemmer = LancasterStemmer()  
  
# Try some stems  
print('drive: {}'.format(stemmer.stem('drive')))  
print('drives: {}'.format(stemmer.stem('drives')))  
print('driver: {}'.format(stemmer.stem('driver')))  
print('drivers: {}'.format(stemmer.stem('drivers')))  
print('driven: {}'.format(stemmer.stem('driven')))
```

OUTPUT

```
drive: driv  
drives: driv  
driver: driv  
drivers: driv  
driven: driv
```

KELİME GRUPLARINI ETİKETLEME

INPUT

```
from nltk.tag import pos_tag  
  
my_text = "James Smith lives in the United States."  
  
tokens = pos_tag(word_tokenize(my_text))  
print(tokens)
```

OUTPUT

```
[('James', 'NNP'),  
( 'Smith', 'NNP'),  
( 'lives', 'VBZ'),  
( 'in', 'IN'),  
( 'the', 'DT'),  
( 'United', 'NNP'),  
( 'States', 'NNPS'),  
( '.', '.' )]
```

Tagler Hakkında Daha Detaylı
Bilgi İçin Görsele Tıklayın



NAMED ENTITY-RECOGNITION (NER)

INPUT

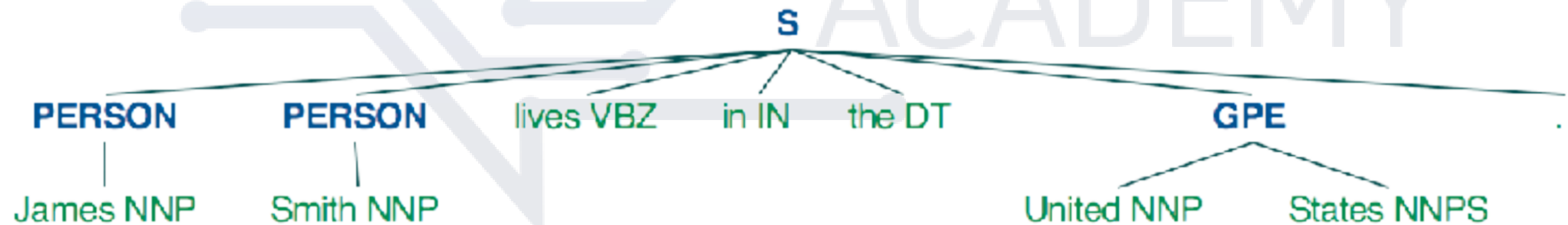
```
from nltk.chunk import ne_chunk

my_text = "James Smith lives in the United States."

tokens = pos_tag(word_tokenize(my_text)) # this labels each word as a part of speech

entities = ne_chunk(tokens) # this extracts entities from the list of words
entities.draw()
```

OUTPUT



KELİME GRUPLARINI TANIMLAMA

Bazı kelimeler **tek başlarına** bir anlam ifade etmezler. Ancak kelime grubu olarak ifade edildiklerinde gerçek anlamları ortaya çıkar.

Hi Mr. Smith! I'm going to buy some vegetables (tomatoes and cucumbers) from the store. Should I pick up some **black eyed peas** as well?

['Black', 'Eyed', 'Peas']



['Black_Eyed_Peas']



BİRLEŞİK KELİMELERİ TANIMLAMA

INPUT

```
from nltk.tokenize import MWETokenizer # multi-word expression
```

```
my_text = "You all are the greatest students of all time."
```

```
mwe_tokenizer = MWETokenizer([('You', 'all'), ('of', 'all', 'time')])
```

```
mwe_tokens = mwe_tokenizer.tokenize(word_tokenize(my_text))
```

```
mwe_tokens
```

OUTPUT

```
['You_all', 'are', 'the', 'greatest', 'students', 'of_all_time', '.']
```


- **Metin Verileri Karmaşıktır!**

- Analiz yapılmadan önce kapsamlı ön işleme adımları uygulanmalıdır.
- NLTK, TextBlob ve spaCy gibi NLP için harika kütüphanelere sahiptir.

- **Metin Verileriyle Çalışırken Birçok Ön İşleme Tekniği Vardır!**

- Analiz için verilerin parçalanması ve düzenlenmesi gereklidir.
- Verileriniz ve analiziniz için en anlamlı teknikleri seçin. Bazı durumlarda burada bahsedilen tüm teknikleri kullanmanıza gerek kalmazken, bazen de daha fazla ön işleme adımına ihtiyacınız olabilir.